Lecture 5 — MapReduce & Data Mining

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

Overview Topics Covered

Part 1 · Similarity Search

Distance and similarity measures, the all-pairs similarity problem, shingling, MinHash signatures, multiple hash functions, and MapReduce implementation.

Part 2 · Clustering

K-Means algorithm, two MapReduce implementations (v1 and v2 with in-mapper combiner), and disk I/O complexity analysis.

Key theme: MapReduce is powerful for data mining at scale, but **iterative algorithms** (like K-Means) expose a fundamental bottleneck — each iteration requires a full disk read/write cycle through HDFS. This motivates later systems like Spark.

1 Similarity as a Common Subroutine

Many large-scale data problems reduce to finding **"similar" objects** — items that are close according to some measure. This comes up repeatedly across data mining.

Duplicate Detection

Documents with similar words — identify near-duplicate web pages, mirror sites, or plagiarised articles so they aren't shown twice in search results.

Recommendation

Customers who purchased similar products — if two users bought the same things, recommend each other's remaining purchases.

User Grouping

Users who visited similar websites — group users with similar browsing patterns for targeted advertising or content personalisation.

All of these problems share the same structure: *given a large collection of objects, find all pairs that are "close" under some similarity measure*. The challenge is doing this efficiently when the collection is huge.

2 Distance & Similarity Measures

To define "near neighbours", we need a **distance function** `d(x, y)`. Lower distance = higher similarity.

Metric	Formula	Key Property	Best for
Euclidean distance	$d(x,y) = \sqrt{\sum(x_i - y_i)^2}$	Straight-line distance in space	Points in continuous space (e.g. coordinates)
Manhattan distance	$d(x,y) = \sum\lvert x_i - y_i\rvert$	Grid/taxicab distance — sum of absolute differences	Grid-like data; robust to outliers in any one dimension
Cosine similarity	$sim(a,b) = (a \cdot b) / (\lvert a\rvert\lvert b\rvert)$	Only considers *direction*, not magnitude. Scaling a or b doesn't change it.	Text documents as word-count vectors; angle between vectors
Jaccard similarity	$sim(A,B) = \lvert A \cap B\rvert / \lvert A \cup B\rvert$	Fraction of shared elements. Jaccard distance = 1 - Jaccard sim.	Sets: documents as shingle sets, user item-sets

Jaccard example: A = {■, ●}, B = {●, ▲}. Intersection = {●} (1 item), Union = {■, ●, ▲} (3 items). Jaccard sim = 1/3. Jaccard distance = 1 - 1/3 = 2/3.

3 The Similar Documents Problem

Given N documents, we want to find all pairs with Jaccard similarity above some threshold (e.g. near-duplicate web pages).

Why naïve pairwise comparison doesn't scale

With N = 1 million documents, naïve all-pairs comparison requires:

$N(N-1)/2 \approx 5 \times 10^{11}$ **comparisons**

Even at 1 million comparisons per second, this would take over 5 days. We need a smarter approach.

The solution is a two-step pipeline:

Step 1: Shingling
Convert each document into a *set* of short overlapping phrases

→

Step 2: MinHash
Compress each set into a short *signature*, preserving similarity

→

Candidate Pairs
Documents with the same signature are *candidate pairs* for near-duplication

4 Step 1 — Shingling

A **k-shingle** (also called a **k-gram**) is a sequence of `k` consecutive tokens that appears in a document. Each document is represented as the *set* of all its k-shingles.

Example: k = 2, document D1 = "the cat is glad"

Sliding a window of size 2 across the tokens:

```
S(D1) = { "the cat", "cat is", "is glad" }
```

Each pair of adjacent words becomes one shingle. The document is now just a *set* of these pairs — and we can use Jaccard similarity to compare sets.

Why sets?

Representing documents as sets of shingles captures the *content* of the document. Two documents with many shared phrases (shingles) will have high Jaccard similarity — which is what we want for detecting near-duplicates.

Choosing k

Larger k → fewer false positives (rarer shingles), but more sensitive to small differences. Typical values: k = 5–10 for web pages. Too small (k=1) means most documents share common words; too large makes shingles unique to each doc.

Formally, the similarity between two documents D1 and D2 is measured as:

```
sim(D1, D2) = |S(D1) ∩ S(D2)| / |S(D1) ∪ S(D2)| (Jaccard similarity of their
        shingle sets)
```

Shingle sets can be represented as a **binary matrix**: rows = all shingles (universe), columns = documents. A 1 in cell (shingle, doc) means that shingle appears in that document. Jaccard similarity = (# rows where both columns are 1) / (# rows where at least one column is 1).

5 Step 2 — MinHash: Overview

With large shingle sets, computing Jaccard similarity directly is still expensive.

e.g. N = 1 million documents. $N(N-1)/2 \approx 5*10^{11}$ comparisons (too slow!)

MinHash compresses each shingle set into a short fixed-length **signature** such that similarity is preserved: highly similar documents tend to produce the same signature.

h(C) should be small enough that the signature fits in RAM

MinHash steps for one hash function h

Given a document's shingle set, e.g. `{ "the cat", "cat is", "is glad" }`:

1. Apply hash function h to each shingle to get an integer:
   `h("the cat") = 12  h("cat is") = 74  h("is glad") = 48`
2. The MinHash signature is the *minimum* hash value:
   `MinHash(D) = min(12, 74, 48) = 12`

This single number is the document's "signature" for hash function h.

Example: comparing two similar documents

Document 1:
`{ "the cat", "cat is", "is glad" }`
Hashes: 12, 74, 48
MinHash = 12

Document 2:
`{ "no cat", "cat is", "is glad" }`
Hashes: 32, 12, 24
MinHash = 12

Same MinHash (12) → these documents are a candidate near-duplicate pair. ✓

Goal

Find a hash function h(·) such that:

1. If sim(S(D1), S(D2)) is high, then with high probability, h(S(D1)) = h(S(D2))
2. If sim(S(D1), S(D2)) is low, then with high probability, h(S(D1)) ≠ h(S(D2))

6 Multiple Hash Functions & Candidate Pairs

A single hash function gives a noisy estimate. In practice, we use **N hash functions** (e.g. N = 100) to generate N MinHash values per document, forming a **signature vector**.

Defining candidate pairs

Two documents are a **candidate pair** if they match on at least *t* out of N hash functions (for some threshold t, e.g. t = 50 out of 100).

The more hash functions that agree, the more confident we are that the documents are genuinely similar. Candidate pairs can then be verified by computing the exact Jaccard similarity — but the number of pairs to verify is far smaller than N(N-1)/2.

Worked example: 3 hash functions, threshold = 2 matches

Document	H_1	H_2	H_3
D1	10	20	23
D2	10	25	23
D3	32	25	15
D4	32	20	25

(D1, D2): match on H_1 and H_3 → candidate pair
(D3, D4): match only on H_1 → not a candidate (only 1 match, need 2)
(D1, D3): no matches → not a candidate

Optional extension — Locality Sensitive Hashing (LSH): A more advanced technique that organises signatures into bands and rows to further narrow down candidate pairs. Not required for this course, but widely used in practice.

7 MapReduce Implementation — Similarity Search

How it works

Map:
- Read each document and extract its shingles (sliding window of k tokens).
- Hash each shingle using the hash function(s) and take the minimum hash value as the MinHash signature.
- Emit `(signature, document_id)`.

Reduce:
- Receive all documents that share a given MinHash signature.
- Generate all candidate pairs from these documents.
- (Optional) Compare each candidate pair directly to verify actual similarity.

```
// MapReduce: Similarity Search via MinHash
        map(doc_id, document):
        shingles = extract_shingles(document, k)
        sig = min(hash(s) for s in shingles)
        emit(sig, doc_id)

        reduce(sig, [doc_id_1, doc_id_2, ...]):
        // all docs sharing this signature are candidate pairs
        for each pair (di, dj) in candidate_pairs:
        emit(di, dj)
```

Why this is efficient: The map phase runs in parallel over all documents with no communication. The shuffle groups documents by signature — only documents with matching signatures ever meet at a reducer. This drastically reduces the number of pairs to consider.

8 Goal of Clustering

Clustering is an unsupervised learning task: given unlabelled data points, group them so that points within a group are similar to each other, and points in different groups are dissimilar.

High intra-cluster similarity

Points inside the same cluster should be close to each other — they share common properties that make them "belong" together.

Low inter-cluster similarity

Points in different clusters should be far apart — the clusters should be clearly separated and distinct from each other.

Applications

Microbiology

Group related genes, proteins, or organisms based on sequence similarity or expression patterns.

Recommendation & Social Networks

Find groups of users with similar tastes for collaborative filtering or community detection.

Search & Information Retrieval

Group similar search results, news articles, or documents by topic for better organisation.

9 K-Means Algorithm

K-Means is the most common clustering algorithm. You specify K (the number of clusters) upfront, and the algorithm iteratively refines K cluster centres until convergence.

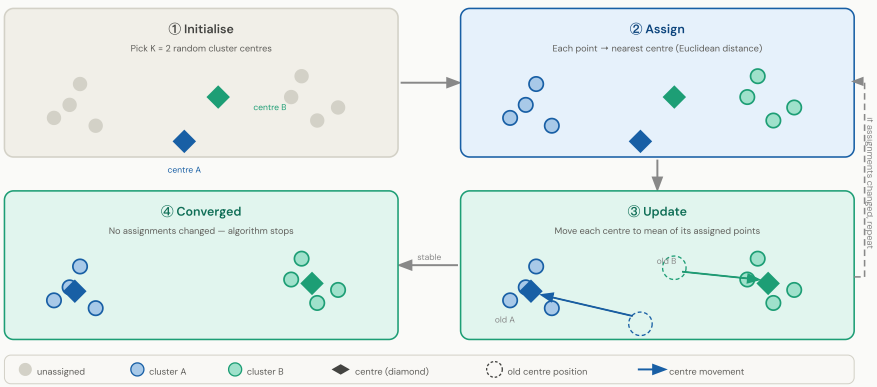
Algorithm steps

Initialisation: Pick K random points as initial cluster centres (also called centroids).

Repeat until no assignments change:

1. **Assignment step:** Assign each data point to the nearest cluster centre (using Euclidean distance).
2. **Update step:** Move each cluster centre to the *mean (average)* position of all points assigned to it.

K-MEANS — ITERATIVE STEPS (K = 2 CLUSTERS)



Limitation of K-Means: The result can depend on the random initialisation. Different starting centres can produce different final clusters. Running K-Means multiple times and picking the best result (lowest total within-cluster distance) is common practice.

10 MapReduce K-Means — Version 1

Each MapReduce job performs **one iteration** of K-Means. The algorithm requires as many jobs as there are iterations until convergence.

How it works

Configure(): Read the current cluster centre positions from a file (stored in HDFS between iterations).

Map: For each data point, compute its distance to every cluster centre (loaded in Configure), and emit `(cluster_id, point)` where `cluster_id` is the nearest centre.

Reduce: For each cluster, receive all points assigned to it. Compute the new cluster centre as the mean (average) of those points. Write the new centre positions back to HDFS.

Map = "Which cluster does this point belong to?"
Reduce = "What's the new average of this cluster?"
Loop = "Repeat until stable"

```
// MapReduce K-Means v1 — one iteration
        configure(): centers = load_centers_from_file() // broadcast to all
          mappers

        map(point_id, point):
        cluster_id = argmin(distance(point, c)
        for c in centers)
        emit(cluster_id, point)

        reduce(cluster_id, [point_1, point_2, ...]):
        new_center = mean([point_1, point_2, ...])
        emit(cluster_id, new_center)
```

Problem **High network traffic**

In v1, every single data point is shuffled from mapper to reducer. With n points each of dimensionality d, running for m iterations, the total disk I/O exchanged between mappers and reducers is:

$O(n \times m \times d)$

This is proportional to the dataset size — shuffling the entire dataset on every iteration is extremely expensive.

Exam question: Why is O(nmd) the answer for v1?
Each of the n points is a d-dimensional vector emitted by the mapper. This happens once per iteration (m iterations total). So total data transferred = n × d × m = O(nmd).

11 MapReduce K-Means — Version 2 (In-Mapper Combiner)

Version 2 uses an **in-mapper combiner** to partially aggregate points *within the mapper* before the shuffle, dramatically reducing network traffic.

Key change: aggregate locally first

Instead of emitting every individual point, each mapper maintains a local data structure H that tracks:

- The **sum** of all points assigned to each cluster
- The **count** of points assigned to each cluster

At the end of the map phase, the mapper emits just **k partial sums** (one per cluster) instead of n individual points.

```
// MapReduce K-Means v2 — in-mapper combiner
        configure():
        centers = load_centers_from_file()
        H = {} // local accumulator: cluster_id → (sum, count)

        map(point_id, point):
        nearest = argmin(distance(point, c) for c in centers)
        H[nearest].sum += point
        H[nearest].count += 1

        cleanup(): // emit once per cluster, not once per
          point
        for cluster_id in H:
        emit(cluster_id, (H[cluster_id].sum, H[cluster_id].count))

        reduce(cluster_id, [(sum_1, count_1), (sum_2, count_2), ...]):
        new_center = sum(all sums) / sum(all counts)
        emit(cluster_id, new_center)
```

Disk I/O with v2 (assuming 1 map task)

Each mapper emits *k* partial sums, each of dimensionality d. With m iterations:

O(k × m × d)

Since k (number of clusters, e.g. k = 10) is typically *much smaller* than n (number of points, e.g. n = 10 million), this is a massive reduction. We no longer pay per-point network cost — only per-cluster.

12 v1 vs v2 — Example Comparison

Step	Version 1 (Naive)	Version 2 (In-Mapper Combiner)
Input (2 Mappers)	Mapper A points: (2,2), (4,4), (6,6) Mapper B points: (3,3), (5,5) All belong to Cluster 1	
Map Output	Mapper A: emit(1, (2,2)) emit(1, (4,4)) emit(1, (6,6)) Mapper B: emit(1, (3,3)) emit(1, (5,5))	Mapper A: sum = (12,12), count = 3 emit(1, ((12,12), 3)) Mapper B: sum = (8,8), count = 2 emit(1, ((8,8), 2))
Data Sent Over Network	5 full points: (2,2), (4,4), (6,6), (3,3), (5,5)	2 aggregated records: ((12,12), 3) ((8,8), 2)
Reducer Input	[(2,2), (4,4), (6,6), (3,3), (5,5)]	[((12,12), 3), ((8,8), 2)]
Reducer Computation	mean = ((2+4+6+3+5)/5, (2+4+6+3+5)/5) = (4,4)	total_sum = (12+8, 12+8) = (20,20) total_count = 3+2 = 5 mean = (20,20)/5 = (4,4)
Key Insight	Sends every point individually	Sends only (sum, count) per mapper

13 v1 vs v2 — Complexity Comparison

Property	K-Means v1	K-Means v2 (in-mapper combiner)
What mapper emits	One record per point → n records per task	One partial sum per cluster → k records per task
Shuffle volume per iteration	**O(nd)** — proportional to dataset size	**O(kd)** — proportional to number of clusters
Total I/O (m iterations)	**O(nmd)**	**O(kmd)**
Memory requirement	Just the k cluster centres	k partial sums + k centres (slightly more)
Correctness	Exact ✓	Exact ✓ (partial sums combined correctly in reducer)

The bigger picture: Even v2 still requires a full MapReduce round-trip (HDFS read → map → shuffle → reduce → HDFS write) for every single K-Means iteration. For large k and many iterations, this is very expensive. This is the fundamental limitation of MapReduce for iterative algorithms — each pass through the data involves full disk I/O. Spark solves this by keeping data in memory across iterations.

13 Exam-Style Q&A

Q: What is a k-shingle and how is it used for document similarity?

A k-shingle (or k-gram) is a sequence of k consecutive tokens (words or characters) from a document. Each document is converted into the *set* of all its k-shingles. Similarity between documents is then measured as the Jaccard similarity of their shingle sets: $|S(D1) \cap S(D2)| / |S(D1) \cup S(D2)|$.

Q: Why can't we just compare all document pairs directly?

With N documents, there are N(N-1)/2 pairs to compare — approximately 5×10^{11} comparisons for N = 1 million. This is computationally infeasible. MinHash + candidate pairs reduces the problem dramatically by only comparing documents that share a MinHash signature.

Q: State the key property of MinHash.

$Pr[h(C_1) = h(C_2)] = \text{Jaccard-Sim}(C_1, C_2)$. The probability that two documents have the same MinHash value under a randomly chosen hash function exactly equals their Jaccard similarity. This allows us to estimate Jaccard similarity by comparing cheap integer hashes instead of full shingle sets.

Q: How does cosine similarity differ from Jaccard similarity?

Cosine similarity measures the angle between two vectors — it only cares about direction, not magnitude (scaling a vector doesn't change it). It's suited for word-count vectors. Jaccard similarity measures set overlap ($|A \cap B|/|A \cup B|$) — it's suited for sparse binary sets like shingle sets.

Q: What are the two steps of the K-Means algorithm?

Initialise K random centres, then repeat: (a) **Assignment** — assign each point to its nearest centre; (b) **Update** — move each centre to the mean of its assigned points. Stop when no assignments change.

Q: Why is MapReduce K-Means v1's disk I/O O(nmd)?

Each of the n points (each d-dimensional) is emitted by a mapper and shuffled to a reducer. This happens once per iteration. Over m iterations: $n \times d \times m = O(nmd)$. The cost scales with the full dataset size per iteration, making it expensive for large n.

Q: How does the in-mapper combiner in v2 reduce I/O to O(kmd)?

Instead of emitting one record per point, each mapper accumulates partial sums per cluster in a local data structure H. At the end of the map phase it emits k partial sums (one per cluster, each d-dimensional). So the shuffle volume per iteration is $k \times d$, and over m iterations: O(kmd). Since $k \ll n$, this is a huge reduction.

Q: Why is MapReduce not ideal for iterative algorithms like K-Means?

Each iteration of K-Means requires a separate MapReduce job. Between jobs, intermediate state must be written to HDFS (disk) and re-read in the next job. This disk round-trip for every iteration is very expensive. Systems like Spark avoid this by keeping data in memory (as RDDs) across iterations.

SUPPLEMENTARY

Supplementary Key Property of MinHash

Fundamental theorem: The probability that two documents have the same MinHash value equals their Jaccard similarity:

$$Pr[\, h(C_1) = h(C_2) \,] = \text{Jaccard-Sim}(C_1, C_2)$$

Intuition for the proof (not required, but useful)

Consider the union of all shingles across both documents. Each shingle has an equal probability of having the smallest hash value. The two documents will have the same MinHash *only if* the shingle with the minimum hash is one that they both share (i.e., in the intersection).

Therefore: Pr[same MinHash] = (size of intersection) / (size of union) = Jaccard similarity. ▪

Why this is powerful: MinHash lets us approximate Jaccard similarity between two huge shingle sets by comparing just a single integer — without storing or comparing the full sets. The approximation improves as we use more hash functions.